Autograder Submission Guideline

This guideline is created with the help of Claude Code. Informed by failures observed in the grading pass of the leaderboard preview, where a substantial fraction of submissions required manual TA rescue.

This is a draft. Please feel free to comment on Piazza if you think there is a better option, as we intend to allow maximum freedom for your RAG setup while guaranteeing that the AG won't run into such chaos for the final submission.

We plan to set up Gradescope autograder to sanity-check for compliance with this guideline.

Be sure to check out the latest release packet on Canvas as some files have been updated during the quarter.

---

0. Submission package

Per the project statement, each team submits a single archive:

```
team_<team_id>.zip
├── main.py                # can add helper modules alongside
├── output.json            # your predictions on the released set
├── report.pdf
├── your_golden_qa.json
├── (optional) requirements.txt
└── repo.zip            # snapshot of your full repo
```

Rules:

1. Exactly one outer `team_<team_id>.zip`. No nested archives beyond the one inner `repo.zip`. No `.tar.gz`, or misspelled file extensions as this will prevent Gradescope from automatically processing your submissions.
2. The autograder grades `main.py` extracted from the outer archive (and uses `repo.zip` for any helper modules it imports). It does NOT walk into arbitrary nested archives.
3. Archive entries must use POSIX paths (forward slashes). Windows backslash entries cause extraction failures.
4. Before zipping, scrub secrets from `repo.zip`. Do not commit `~/api-key.txt`, `.env`, personal cloud credentials, or any third-party tokens into your repo snapshot.
   Hardcode your API key into the code ***ONLY IF*** you need to use external services.

How output.json is used

`output.json` in your archive is your predictions on the **released (public)** set. The autograder computes your released-set scores (retrieval; the public 3-metric thru LLM judge) without re-running your pipeline on the released set.

For the **hidden** set, the autograder re-runs `main.py` from scratch. Both released-set and hidden-set retrieval and generation scores will be included in the final published results.

---

1. CLI

We're adding some more requirements to the statement's.

`main.py` should be invokable as:

```
python main.py                                                  \
          --input <path>            --output <path>             \
          --corpus-dir <path>       --apikey-txt <path_to_txt>
\
          --generation-model <name>
```

Argument semantics:

flag	type	meaning
--input	path	JSON file of [{question_id, question}, ...]
--output	path	Where to write the predictions JSON
--corpus-dir	path	Root of the .rst corpus (the released sourcedocs/ layout)
--apikey-txt	path	Path to the text file of the gateway API key
--generation-model	string	Gateway model id (e.g. claude-sonnet-4-6, api-gpt-oss-120b) for generating answers to queries; in case 429 error is encountered for some models. Used as fallback to generate a working output for calculating retrieval scores.

Rules:

1. All five flags must be accepted. Use `argparse` (or equivalent) — do NOT rely on positional `sys.argv[1] / sys.argv[2]`. If you are using an external vector DB, your script can ignore what is passed for --corpus-dir (but will still need to accept it).
2. You may define additional flags with sensible defaults, but don't expect the autograder to pass them.
3. You should not require any environment variable to be set in order to run. The autograder won't set the variables on your behalf.

---

2. Runtime assumptions

1. The RapidFire AI dispatcher is **already running** when your code starts. DO NOT start, stop, or restart the dispatcher inside `main.py`.
2. If you use RapidFire's `run_evals`, remember to convert its output into the schema in section 4. The autograder does NOT consume RapidFire's internal

artifacts directly. Empty or malformed predictions from `run_evals` will hinder LLM judge's processing.

3. Submit **a single inference configuration**. Do NOT launch multi-config sweeps or experiments inside `main.py`.
4. The Python environment is the course `cse234` conda env. CPU only — no GPU on the autograder host.
5. **Do not compute eval metrics in your script**. This will slow down the script and may cause errors if you attempt to call LLM judge (all inputs to `main.py` will be stripped of the `reference_answer` and `source` field).

2a. External services policy

We intend to allow external services (managed vector DBs, and/or embedding APIs).

You will need to declare the usage of these in the report, and hardcode your API keys in the code.

---

3. Dependencies (requirements.txt)

If your submission needs packages beyond what is preinstalled in `cse234`, list them in `requirements.txt` at the root of your archive. The autograder will try to install **only** what is in that file.

Constraints on what to put in `requirements.txt`:

1. Keep the list **minimal**. ~~Do not pin or add packages that are already in `cse234`.~~ You may include packages that are already in the course environment, and the autograder will compare and determine which packages need to be installed.
2. **Do NOT use `langchain-anthropic` or `langchain-google-genai`.** This is intended as a measure to minimize the required packages in the environment. The TritonAI gateway supports both OpenAI- and Anthropic-style API requests, though.

3. Do not pin packages to versions that conflict with the `cse234` baseline (e.g. a different `langchain` major). Such pins will be rejected at install time.

---

4. Output schema

A JSON file with fields defined as in the statement. Be sure not to write synonyms!

- `question_id` key with an integer value
- `answer` key with a string value.
- `retrieved_context` key with a string value: the full concatenated context text that was passed to the generator LLM
- `sources` key with a list of one or more source evidence locations as value, each specified as:
  `{"file": <filename>, "lines": [<startline>, <endline>]}`

---

5. Recommended self-check

Before submitting, run end-to-end with the released set:

```
python main.py \
  --input  golden_qa_pairs.json \
  --output predictions.json \
  --corpus-dir sourcedocs \
  --apikey-txt ~/api-key.txt \
  --generation-model "api-gpt-oss-120b"
```

Then verify:

1. `output.json` exists and is a valid json.
2. Every record in the json has the four required fields (section 4) with **exact** names.

3. The `sources` and `retrieved_context` field in the json is non-empty for at least the questions your retriever hits.
4. `metrics/evaluate_retrieval.py` runs to completion against the output.
5. `metrics/run_judge.py` runs to completion against the output.
6. `python -c "import ast; ast.parse(open('main.py').read())"` succeeds
   (catches errors such as stray merge-conflict markers and/or syntax errors).

If your program runs and the six steps are passed, your submission should be guideline-compliant.